

End-to-End Big Data Sentiment Analysis on Databricks

Hariprakash Karthikeyan

Abstract—With the growing volume of unstructured textual data generated by social media platforms, scalable sentiment analysis pipelines are essential for extracting actionable insights in near real time. This project presents the design, implementation, and performance analysis of an end-to-end distributed sentiment analytics pipeline using Apache Spark on Databricks. The pipeline ingests raw textual data, applies sentiment prediction outputs from pretrained language models, normalizes heterogeneous label formats, and produces analytics-ready Delta tables. Model evaluation is performed directly on curated Gold-layer data, and key performance metrics are logged using MLflow to ensure reproducibility. In addition, application-level analysis is conducted to identify sentiment trends across extracted entity mentions. Finally, the pipeline’s execution characteristics are analyzed using the Spark UI across five optimization dimensions: spill, skew, shuffle, storage, and serialization. Results demonstrate efficient execution with minimal overhead, balanced task distribution, and effective use of Spark’s memory and storage mechanisms.

I. INTRODUCTION

Sentiment analysis plays a critical role in understanding public opinion, customer feedback, and brand perception across digital platforms. While modern transformer-based language models provide strong predictive performance, deploying these models in real-world analytics workflows requires scalable data processing infrastructure capable of handling large volumes of text efficiently.

Apache Spark has emerged as a standard framework for large-scale data processing due to its distributed execution model, in-memory computation, and rich ecosystem for structured and unstructured data analysis. However, achieving performant Spark applications requires careful consideration of pipeline design, data layout, and execution behavior.

The objective of this project is twofold. First, it aims to build a production-style sentiment analytics pipeline that transforms raw model outputs into analytics-ready datasets suitable for evaluation and downstream insights. Second, it seeks to analyze the runtime performance of this pipeline using Spark UI metrics, identifying whether key performance bottlenecks such as memory spill, data skew, excessive shuffle, inefficient storage access, or serialization overhead are present.

By combining functional correctness with execution-level analysis, this project demonstrates both analytical rigor and systems-level understanding of distributed data processing.

II. DATA PIPELINE ARCHITECTURE

The pipeline follows a layered Delta Lake architecture, consisting of ingestion, transformation, and analytics stages. Raw sentiment prediction outputs are stored in Delta format

and progressively refined through Spark SQL and PySpark DataFrame transformations.

A key design challenge addressed in this pipeline is the normalization of heterogeneous sentiment labels. Model outputs may use different label conventions, such as `asLABEL_0`, `LABEL_1`, and `LABEL_2`, or abbreviated forms such as `NEG`, `NEU`, and `POS`. These are mapped to standardized human-readable sentiment categories: negative, neutral, and positive. This normalization ensures consistency across evaluation and application-level analytics.

The final Gold layer contains record-level ground truth sentiment labels alongside predicted sentiment labels and extracted entity mentions. This table serves as the authoritative source for both model evaluation and business-oriented analytics.

III. DATASET DESCRIPTION

The dataset used in this project consists of application-level textual records enriched with sentiment predictions generated by a pretrained language model. Each record contains the original text content, a predicted sentiment label, and extracted entity mentions used for downstream analytics. The data is stored in Delta Lake format and processed using Apache Spark on Databricks to support scalable transformation and evaluation.

The total volume of data processed during pipeline execution is on the order of hundreds of megabytes, with over one hundred thousand records distributed across multiple Spark partitions. While the dataset size is moderate, the heterogeneity of label formats and the need for aggregation across multiple dimensions motivate the use of a distributed processing framework rather than local, single-node execution.

Key attributes utilized in the pipeline include the predicted sentiment output, normalized sentiment labels, ground-truth sentiment labels for evaluation, and extracted mentions used for application-level analysis. The structured layout of the Gold table enables both record-level model evaluation and higher-level aggregation without requiring repeated preprocessing.

IV. MODEL EVALUATION METHODOLOGY

Model evaluation is performed directly on the Gold Delta table to ensure alignment with production-grade data. Records are filtered to focus on binary sentiment classification by excluding neutral cases, enabling clearer interpretation of precision, recall, and F1-score.

After filtering, Spark DataFrames are converted to Pandas DataFrames for metric computation using standard evaluation utilities. Precision, recall, and F1-score are computed

using macro averaging to treat positive and negative classes equally, preventing class imbalance from biasing results.

A confusion matrix is generated to visualize classification performance and is saved as an artifact. All evaluation metrics, parameters, and artifacts are logged using MLflow, along with metadata such as the Gold table version. This ensures experiment traceability and reproducibility across pipeline runs.

V. DESIGN DECISIONS AND TRADE-OFFS

Several design decisions were made to balance performance, clarity, and extensibility of the pipeline. Delta Lake was selected as the storage format to enable ACID guarantees, versioned data access, and compatibility with Spark SQL operations. This choice simplifies evaluation reproducibility by allowing metrics to be associated with specific Gold table versions.

Sentiment label normalization was performed early in the pipeline to reconcile heterogeneous model outputs into a consistent representation. This design choice avoids repeated conditional logic in downstream evaluation and analytics, improving maintainability at the cost of a small upfront transformation.

For model evaluation, neutral sentiment instances were excluded to focus on binary classification performance. While this simplifies interpretation of precision, recall, and F1-score, it intentionally trades off completeness for clarity in evaluating model discrimination between positive and negative sentiment.

Finally, the conversion from Spark DataFrames to Pandas was deferred until the final stages of analysis. This approach minimizes driver memory usage while still enabling the use of mature Python-based evaluation and visualization libraries.

VI. APPLICATION LEVEL ANALYTICS

Beyond model evaluation, the pipeline supports application-level sentiment analysis by aggregating sentiment counts across extracted entity mentions. Mentions are grouped by sentiment category, and total counts are computed to identify the most frequently discussed entities.

The analysis produces ranked lists of top mentions by positive and negative sentiment. These results are visualized using bar charts to provide intuitive insights into sentiment distribution. This component demonstrates how curated Gold-layer data can be repurposed for exploratory analysis and reporting without modifying the underlying pipeline.

VII. SPARK PERFORMANCE ANALYSIS

To assess the efficiency of the pipeline, execution metrics were examined using the Spark UI. Performance was evaluated along five key dimensions: spill, skew, shuffle, storage, and serialization.

Spill: No significant memory spill was observed. Executor disk usage remained minimal, and there were no spill indicators in the stage metrics. Garbage collection time was

negligible relative to task execution time, indicating sufficient memory allocation.

Skew: Task durations across stages were uniform, with no long-running stragglers. All executors exhibited balanced workload distribution, suggesting even partition sizes and effective parallelism.

Shuffle: Shuffle operations incurred moderate network I/O, corresponding to wide transformations such as group-by aggregations. However, shuffle read and write volumes remained small relative to total input size, indicating efficient data movement.

Storage: Data was stored and read using the Parquet format, resulting in efficient disk access patterns. A substantial fraction of reads were served from the local I/O cache, reducing repeated disk access and improving performance.

Serialization: Serialization overhead was minimal. The pipeline relies primarily on native Spark SQL and DataFrame operations, with limited Python-side logic. No serialization bottlenecks were evident in executor or stage metrics..

VIII. SCALABILITY AND PRODUCTION CONSIDERATIONS

The pipeline is designed to scale horizontally with increasing data volume by leveraging Spark’s distributed execution model. As dataset size grows, transformations such as filtering, aggregation, and joins can be parallelized across additional executors with minimal changes to application logic.

Potential scalability considerations include tuning the number of shuffle partitions for large group-by operations and avoiding driver-side data collection when working with significantly larger datasets. For production deployment, entity-level aggregation and sentiment summarization could be implemented entirely within Spark to eliminate Pandas-based bottlenecks.

The use of Delta Lake also enables incremental updates and time-based partitioning, which would allow the pipeline to process streaming or near-real-time data efficiently. Integration with Spark Structured Streaming could further extend the pipeline to support continuous sentiment monitoring.

IX. LIMITATIONS AND FUTURE IMPROVEMENTS

Despite its effectiveness, the current pipeline has several limitations. The reliance on `toPandas()` for evaluation and visualization restricts scalability beyond moderate data sizes. Additionally, the quality of mention-based analytics depends on the accuracy of entity extraction, which is treated as a fixed upstream process in this project.

The Spark UI analysis reflects performance under the current data scale and cluster configuration and may not capture bottlenecks that emerge at significantly higher volumes. Neutral sentiment exclusion also limits the scope of evaluation to binary classification.

Future improvements include fully Spark-native evaluation metrics, enhanced skew detection for large-scale aggregations, integration of streaming data sources, and automated

performance monitoring dashboards. Expanding model comparison and experiment tracking within MLflow would further strengthen reproducibility and analytical rigor.

X. RESULTS

The pipeline successfully produced analytics-ready sentiment data while maintaining efficient execution characteristics. Evaluation metrics demonstrated stable classification performance, and confusion matrix analysis revealed balanced predictive behavior between positive and negative classes.

From a systems perspective, Spark UI analysis confirmed that the pipeline operates within optimal execution bounds. The absence of memory spill, lack of skew, controlled shuffle volume, efficient storage access, and minimal serialization overhead collectively indicate a well-designed and scalable Spark application.

XI. REFLECTIONS, LEARNINGS AND FUTURE DIRECTIONS

This project was designed not only to produce sentiment analytics but also to explore how large-scale data processing pipelines behave under real execution conditions. A key learning from this work was the importance of treating performance analysis as a first-class component of pipeline design rather than an afterthought. By examining execution metrics in the Spark UI, it became clear how design decisions at the DataFrame and storage level directly influence runtime behavior.

One of the most valuable insights gained was understanding how efficiently Spark can operate when transformations are structured carefully. The absence of memory spill and skew reinforced the importance of balanced partitioning and minimizing unnecessary wide transformations. Observing shuffle behavior and cache utilization through the Spark UI also highlighted how even moderate changes in aggregation logic or data layout can significantly affect execution performance.

From an architectural perspective, this project emphasized the benefits of layered data modeling using Delta Lake. Separating raw ingestion from curated Gold-layer analytics simplified both evaluation and application-level analysis. It also enabled reproducibility through versioned data access, which is essential for maintaining trust in analytical results.

There are several directions in which this project could be extended. From a scalability standpoint, driver-side Pandas operations could be replaced with fully Spark-native evaluation and visualization workflows to support significantly larger datasets. Entity extraction and sentiment aggregation could also be integrated into a streaming architecture using Spark Structured Streaming, enabling near-real-time sentiment monitoring. Additionally, expanding model comparison within MLflow would allow systematic evaluation of multiple sentiment models over time.

Overall, this project strengthened my understanding of distributed data processing, performance tuning, and the practical trade-offs involved in building analytics pipelines

at scale. It reinforced the idea that effective data engineering requires both functional correctness and an awareness of how systems behave in production. These lessons are directly applicable to real-world data science and data engineering roles where scalability, observability, and reproducibility are critical.